

Spring Security en Spring MVC

Sesión 1: Autenticación, Criptografía, Cookies,
Sesiones y Filter Chain

Computación en Internet II — Universidad Icesi

Ruta de Aprendizaje — Sesión 1

Bloque 1: Identidad & Criptografía



01. Autenticación & Criptografía

AuthN vs AuthZ, Encoding vs Encryption vs Hashing, Rainbow Tables y BCrypt.



02. Cookies, Sesiones & DevTools

Protocolo Stateless, cabeceras Set-Cookie, flags de seguridad, Tomcat RAM y Maven.

Live Demo 1: Inspección de Cookies, flags de seguridad y JSESSIONID en DevTools.

Bloque 2: Filtros & Login Propio



03. Cadena de Filtros (Filter Chain)

DelegatingFilterProxy, FilterChainProxy, filtros cardinales y ExceptionTranslation.



04. UserDetails & Custom Login

Contratos UserDetails/UserDetailsService, SecurityConfig, LoginController y login.html.

Live Demo 2: Configuración de SecurityFilterChain, login personalizado y depuración en consola.

01. Autenticación, Criptografía y Hash de Contraseñas

Fundamentos teóricos, modelos de seguridad, codificación, encriptación y hashes.

Fundamento: Autenticación vs. Autorización

Autenticación (AuthN — ¿Quién eres?)

Es el proceso de **verificar la identidad proclamada** por el usuario a través de credenciales (correo, contraseña, OTP o biometría).



Primer Muro de Control

Si las credenciales no son válidas, el usuario no ingresa al sistema.



Punto de Falla: 401 Unauthorized

Redirige automáticamente al formulario `/login` o devuelve error 401.

Metáfora: El *pasaporte digital* verificado por la autoridad migratoria.

Autorización (AuthZ — ¿Qué puedes hacer?)

Es la **comprobación de privilegios, roles y permisos** sobre un recurso específico (ej. `/roles-mvc`).



Ocurre Tras la Autenticación

Sólo un usuario ya autenticado puede someterse a evaluación de permisos.



Punto de Falla: 403 Forbidden

Acceso denegado si el usuario no posee el rol requerido (ej. estudiante vs admin).

Metáfora: La *tarjeta llave* que abre únicamente ciertas puertas autorizadas.

Modelo Conceptual: Pasaporte vs Llave de Acceso

Autenticación vs. Autorización

Los dos pilares fundamentales de la seguridad a nivel de aplicación (Application-Level Security)

FASE 1: IDENTIDAD

Autenticación (Authentication)

Pregunta clave: ¿Quién eres? (Who are you?)



USER IDENTITY
Principal: `miUsuario`
Credentials: `.....` (Verificado)
AUTHENTICATED

MECANISMOS DE VALIDACIÓN

HTTP Basic

Form Login

JWT / OAuth2

Resultado: Se crea el objeto **Authentication** en memoria.

OK

FASE 2: PERMISOS

Autorización (Authorization)

Pregunta clave: ¿Qué puedes hacer? (What can you do?)

ROLES (WHO YOU ARE)

- `ROLE_ADMIN`
- `ROLE_MANAGER`
- `ROLE_CLIENT`

AUTHORITIES (CAN DO)

- `READ_PRODUCTS`
- `WRITE_ORDER`
- `DELETE_RECORD`

PUNTOS DE VERIFICACIÓN

Filtros HTTP (Web)

@PreAuthorize (Métodos)

Resultado: Permite (**200 OK**) o deniega (**403 Forbidden**).

Tres Paradigmas de Transformación de Datos

CODIFICACIÓN (ENCODING)

1. Codificación (Encoding)

Propósito: Adaptar datos a formatos seguros para transporte en protocolos (ej. `Base64`, `URL-Encode`, `Hex`).

Reversibilidad: **100% Reversible sin secreto.**

Seguridad: Cero confidencialidad. Nunca usar para proteger contraseñas.

CIFRADO (ENCRYPTION)

2. Cifrado (Encryption)

Propósito: Proteger confidencialidad en tránsito/reposo con algoritmos simétricos/asimétricos (ej. `AES`, `RSA`).

Reversibilidad: **Reversible con clave secreta.**

Seguridad: Si la clave se filtra, todos los datos quedan expuestos.

HASHING CRIPTOGRÁFICO

3. Hashing Criptográfico

Propósito: Generar un resumen matemático irreversible de longitud fija (ej. `BCrypt`, `SHA-256`).

Reversibilidad: **Estrictamente Unidireccional.**

Seguridad: Estándar de oro para persistir contraseñas de usuarios.

Comparación Visual: Encoding vs Encryption vs Hashing

Codificación, Encriptación y Hashing

Comparativa de los 3 procesos de transformación de datos (Laurentiu Spilca — Spring Security)

FORMATO

1. Codificación (Encoding)

Cambio de formato para transporte / lectura.

Ej: Base64, ASCII, URL Encoding

Sin secretos: utiliza estándares públicos.

INPUT (ORIGEN)

"admin:123"

BASE64

OUTPUT (ENCODED)

"YWRtaW46MTIz"

100% REVERSIBLE

PROPIEDAD CLAVE

Cero Confidencialidad

No protege el secreto.

Cualquiera puede decodificarlo de manera inmediata sin claves.

CONFIDENCIALIDAD

2. Encriptación (Encryption)

Cifrado reversible protegido por una clave.

Ej: AES-256, RSA, TLS/HTTPS

Protege datos en tránsito y en reposo.

PLAINTEXT

"Tarjeta 4509"

+ CLAVE

CIPHERTEXT (HEX)

"8f4e2a1b9c..."

REQUIERE SECRET KEY

PROPIEDAD CLAVE

Bidireccional con Llave

Existe función inversa.

Solo quien posee la clave secreta puede recuperar el dato.

UNIDIRECCIONAL

3. Funciones Hash (Hashing)

Trampa matemática: irreversible por diseño.

Ej: BCrypt, Argon2, PBKDF2

Uso obligatorio para contraseñas.

PASSWORD PLANA

"miClave123"
+ Salt Aleatorio

HASH

DIGEST (60 CHAR)

"\$2a\$10\$wK...9xZ"
Longitud siempre fija

IRREVERSIBLE (-X-)

VERIFICACIÓN SEGURA

Nunca se desenscripta

Se calcula el hash de la contraseña ingresada y se compara con matches().

La Amenaza: Rainbow Tables y Fuerza Bruta

Vulnerabilidad de Hashes Simples

En un hashing determinista tradicional (ej. MD5 o SHA-256 puro), la misma clave produce siempre el mismo hash: `hash("123456") = e10adc394...`



Tablas Arcoíris (Rainbow Tables)

Tablas masivas de búsqueda precalculadas con millones de pares contraseña → hash. Un atacante busca el hash robado y obtiene la clave en milisegundos sin calcular nada.

Fuerza Bruta Acelerada con GPU

Los algoritmos SHA fueron diseñados para ser *extremadamente veloces* al validar gigabytes de datos, haciéndolos letales para contraseñas.



> 50.000 Millones de Hashes/seg

Un cluster moderno de GPUs puede probar miles de millones de combinaciones por segundo, crackeando claves de 8 caracteres en minutos.

Defensas: Salt Aleatorio y Costo Adaptativo

Defensa 1: Salt Criptográfico Aleatorio

Secuencia aleatoria de 16+ bytes (128 bits) generada dinámicamente y concatenada con la contraseña: `Hash(password + salt)`.



Hashes Únicos por Usuario

Dos usuarios con la misma contraseña ("admin123") tendrán hashes completamente distintos en la BD.



Inmunidad Total a Rainbow Tables

Hace que las tablas precalculadas sean 100% inútiles, obligando al atacante a crackear usuario por usuario.

Defensa 2: Costo Adaptativo (Work Factor)

Uso de funciones *deliberadamente lentas* e intensivas en CPU/RAM (ej. **BCrypt**, **Argon2**).



Parámetro de Costo (2^{cost})

Con costo 10 ($2^{10} = 1,024$ rondas), calcular un hash toma ~100 ms en CPU.



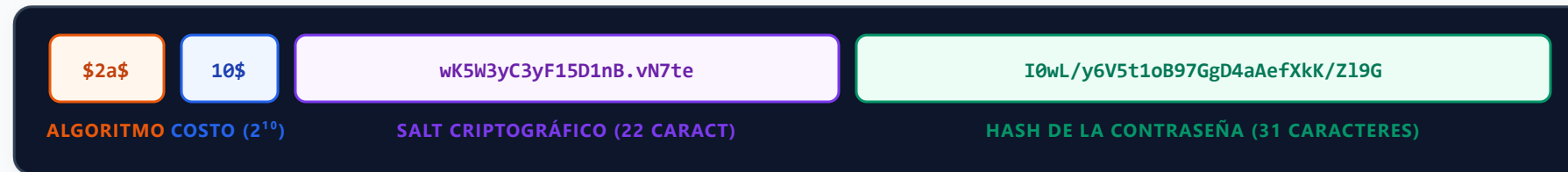
Ataque Inviabile en Tiempo

Para un usuario 100 ms es instantáneo; para un atacante probando mil millones de claves equivale a **siglos de cómputo**.

Anatomía del Hash BCrypt (RFC 2898)

Anatomía del Hash BCrypt (60 Caracteres)

Estructura interna generada y verificada automáticamente por BCryptPasswordEncoder



1. ALGORITMO & WORK FACTOR

\$2a\$ / \$2b\$ / Costo 10

- **\$2a\$**: Versión de BCrypt.
- **Work Factor (10)**: $2^{10} = 1,024$ rondas de cálculo en la función Blowfish.
- Ralentiza ataques de fuerza bruta incluso con GPUs masivas.

2. SALT CRIPTOGRÁFICO (128 BITS)

22 Chars Base64 Custom

- Se genera de forma **100% aleatoria** en cada codificación.
- Dos usuarios con la misma clave (ej. "123456") tendrán hashes distintos.
- **Inmunidad total** a Rainbow Tables.

3. COMPARACIÓN EN LOGIN

matches(raw, hash)

- BCrypt extrae el Salt del hash BD.
- Hashea la clave enviada con ese Salt.
- Compara los 31 caracteres finales en **tiempo constante** para evitar ataques de temporización (timing attacks).

- **Regla de Oro**: El hash almacenado **ya incluye su propio Salt y Costo** ; por eso la base de datos no requiere una columna adicional para guardar el Salt.

02. Cookies, Sesiones HTTP y Seguridad Web

Protocolo stateless, cabeceras Set-Cookie, flags de seguridad, Tomcat RAM y dependencias Maven.



HTTP Sin Estado



Reto de Identidad



Cookies & Headers

HTTP es Stateless por Diseño



El Protocolo HTTP es Sin Estado (Stateless)

Por diseño original, cada petición Request/Response es completamente aislada. El servidor procesa el request y olvida al cliente de inmediato.



El Gran Desafío de las Aplicaciones Web

Si un usuario se autentica en la Petición 1 (/login), ¿cómo sabe el servidor que la Petición 2 (/usuarios) proviene exactamente del mismo usuario?



La Solución: Estado Gestionado en la Capa Web

Se requiere un mecanismo en las cabeceras HTTP para transportar un testigo o identificador en cada transacción.

Anatomía y Flujo de una Cookie HTTP

Flujo de Cabeceras HTTP

Una **Cookie** es un par clave=valor enviado por el servidor que el navegador almacena y adjunta automáticamente en cada request futuro al mismo dominio.



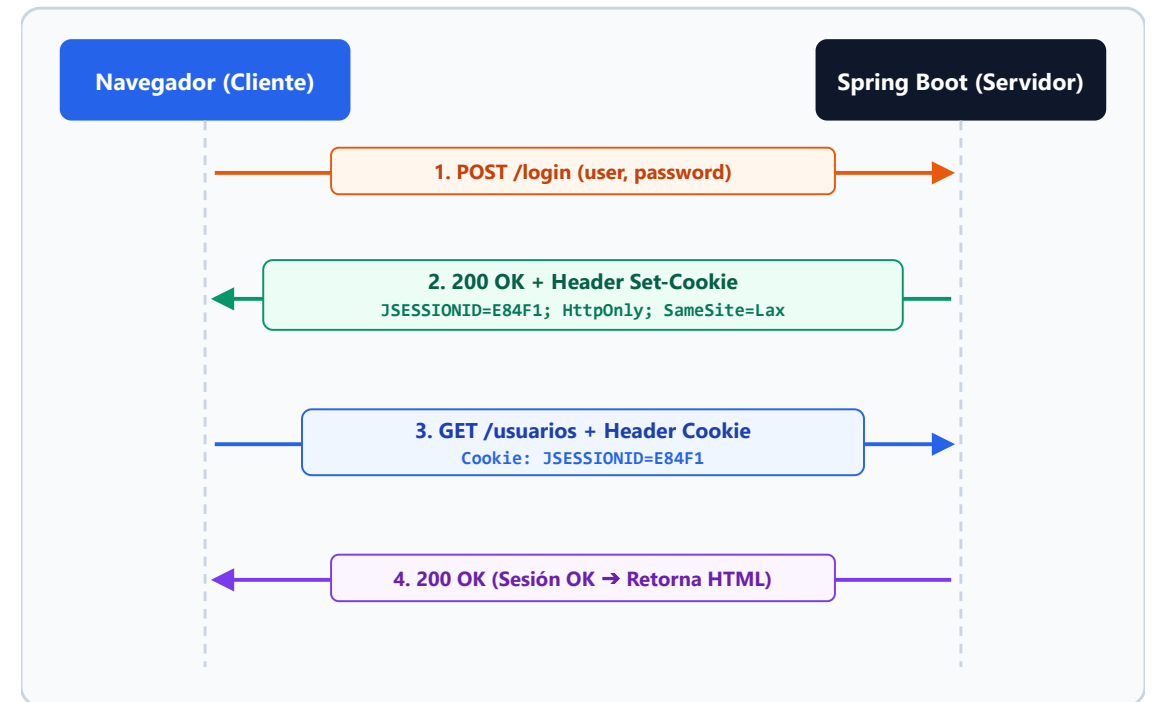
Set-Cookie (Servidor → Cliente)

El servidor responde ordenando al navegador guardar la cookie en su almacén local.



Cookie (Cliente → Servidor)

En cada request posterior, el navegador inyecta el header automáticamente sin intervención de JS.



Atributos Críticos de Seguridad en Cookies

HTTPONLY

HttpOnly

Impide que scripts de JavaScript (ej. `document.cookie`) lean la cookie. Es la *defensa primordial contra ataques XSS*.

SECURE

Secure

Instruye al navegador a transmitir la cookie *únicamente sobre conexiones HTTPS cifradas*, previniendo intercepciones Man-in-the-Middle.

SAMESITE

SameSite (Strict / Lax)

Controla si la cookie se envía en peticiones cross-site iniciadas desde otros dominios. *Protege contra ataques CSRF*.

MAX-AGE & PATH

Max-Age & Path

Max-Age / Expires define el tiempo de vida en segundos antes de expirar. **Path** restringe qué rutas del servidor reciben la cookie.



Tomcat RAM



JSESSIONID



Datos Protegidos

Arquitectura de HttpSession y JSESSIONID



Almacén de Estado en Memoria (RAM)

El contenedor de Servlets (Tomcat) reserva un objeto en memoria (`HttpSession`) para guardar la información del usuario autenticado (roles, username, atributos).



El Identificador de Sesión (JSESSIONID)

Para asociar peticiones al usuario correcto, el servidor genera un ID aleatorio de alta entropía (`JSESSIONID`) y se lo entrega al cliente en una cookie.



Seguridad por Ocultación de Datos

El navegador nunca almacena los datos sensibles ni los roles; solo posee la clave de acceso (el Session ID).

Dependencias Maven del Proyecto

Starters de Seguridad para Spring Boot



spring-boot-starter-security

Activa automáticamente la infraestructura de seguridad, la cadena de filtros por defecto, generador de claves y protección CSRF.



thymeleaf-extras-springsecurity6

Dialecto de Thymeleaf para evaluar atributos `sec:authorize` y `sec:authentication` en el HTML.

Configuración: Agrega estas dos dependencias en el archivo `pom.xml` del proyecto `springboot/` antes de ejecutar los Live Demos.

```
<!-- springboot/pom.xml -->
<dependencies>
  <!-- Starter Núcleo de Seguridad -->
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-security</artifactId>
  </dependency>

  <!-- Integración con Vistas Thymeleaf -->
  <dependency>
    <groupId>org.thymeleaf.extras</groupId>
    <artifactId>thymeleaf-extras-springsecurity6</artifactId>
  </dependency>
</dependencies>
```

Live Demo 1: Inspección de Cookies en DevTools

Actividad en Vivo: Explorando el Almacén Web



1. Abrir Herramientas de Desarrollador

Presiona `F12` o `Ctrl + Shift + I` en tu navegador web.



2. Navegar a Application > Storage > Cookies

Selecciona el origen (`http://localhost:8080`) para inspeccionar la tabla de cookies.



3. Analizar las Banderas de Seguridad

Verifica si las columnas `HttpOnly` , `Secure` y `SameSite` tienen checks activos.



4. Experimento: Borrar JSESSIONID

Elimina la cookie manualmente y recarga la página. El servidor pierde la sesión y redirige al login.

// Headers en DevTools Network

HTTP/1.1 200 OK

Set-Cookie:

JSESSIONID=8A5C1E9D4B2...;

Path=/springboot-api;

HttpOnly;

SameSite=Lax

03. Cadena de Filtros, UserDetails y Login Personalizado

DelegatingFilterProxy, SecurityFilterChain, contratos UserDetails/UserDetailsService, LoginController y vista custom.

Del Contenedor Servlet a Spring Context

El Puente: DelegatingFilterProxy

Los servlets y filtros tradicionales son gestionados por el contenedor web (Tomcat), que *no conoce el ApplicationContext de Spring*.



DelegatingFilterProxy

Filtro estándar registrado en el Servlet Container. Su único propósito es interceptar la petición y delegarla a un Bean de Spring.



FilterChainProxy

Bean maestro de Spring Security. Coordina y ejecuta una o más listas ordenadas de filtros de seguridad (`SecurityFilterChain`).

¿Por qué este diseño desacoplado?

Permite que todos los filtros de seguridad se beneficien de la inyección de dependencias de Spring (IoC), accediendo a servicios, repositorios y configuraciones de forma nativa.



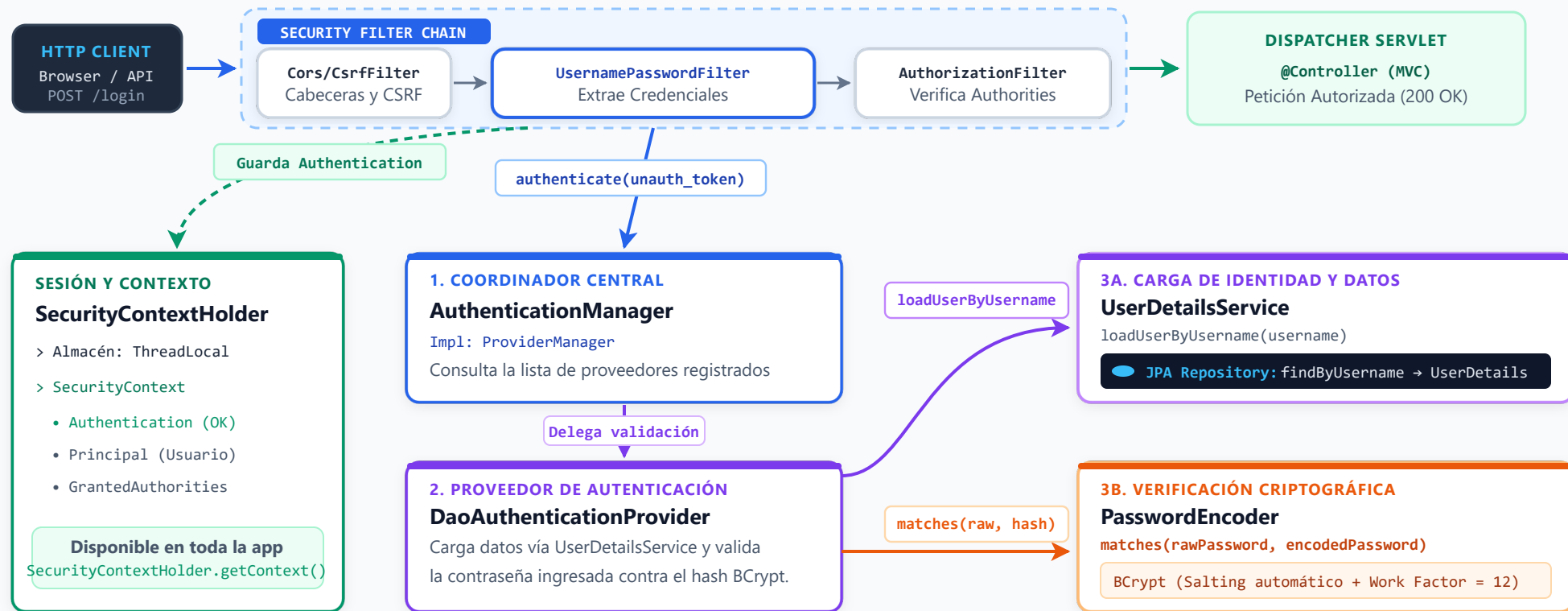
El Primer Muro de Defensa

Toda petición debe superar la cadena de filtros en orden estricto. Si es rechazada (401/403), **jamás toca los controladores de tu aplicación**.

Arquitectura de la Security Filter Chain

Arquitectura Interna de Spring Security

Flujo de autenticación: SecurityFilterChain, AuthenticationManager, Providers y UserDetails



● Flujo: HTTP → AuthFilter → AuthManager → DaoAuthProvider → UserDetailsService (BD) → PasswordEncoder (BCrypt) → SecurityContext → DispatcherServlet (OK)

Filtros Cardinales de la Cadena

CONTEXT PERSISTENCE

SecurityContextHolderFilter

Restaura el `SecurityContext` desde la `HttpSession` al inicio del hilo y garantiza su persistencia al finalizar la petición.

AUTHENTICATION FILTER

UsernamePasswordAuthFilter

Intercepta peticiones de login (por defecto `POST /login`), extrae `username/password` y delega al `AuthenticationManager`.

CSRF PROTECTION

CsrfFilter

Inspecciona que cada petición mutadora (POST, PUT, DELETE) contenga un token CSRF válido generado por el servidor para evitar falsificaciones.

AUTHORIZATION FILTER

AuthorizationFilter

El guardián final de la cadena. Evalúa si el usuario autenticado posee los roles o permisos exigidos por las reglas de URL antes de pasar al Controller.



Captura Errores



401 → Login



403 → Prohibido

Manejo de Excepciones de Seguridad



Captura de Excepciones de Seguridad

Este filtro no realiza validaciones directas; envuelve la ejecución de los siguientes filtros y captura cualquier error de seguridad lanzado en la cadena.



AuthenticationException → Redirección o 401

Si el usuario es anónimo y la ruta requiere autenticación, guarda la petición solicitada y *lo redirige a la página de login* (o retorna `401 Unauthorized` en APIs).



AccessDeniedException → Error 403 Forbidden

Si el usuario ya está autenticado pero *no tiene los roles requeridos*, cancela la petición y responde con código `403 Forbidden`.

Contratos Fundamentales: UserDetails vs UserDetailsService

1. UserDetails (La Identidad)

Representa la entidad del usuario ante Spring Security. Modela credenciales y estado de cuenta:



getUsername() / getPassword()

Retorna el identificador principal (email) y el hash BCrypt.



getAuthorities()

Colección de roles y permisos (`GrantedAuthority`).



Flags de Cuenta

`isEnabled()`, `isAccountNonLocked()`, `isCredentialsNonExpired()`.

2. UserDetailsService (El Proveedor)

Interfaz funcional con un único método que localiza al usuario según su nombre o email:

```
public interface UserDetailsService {  
    UserDetails loadUserByUsername (String u)  
        throws UsernameNotFoundException;  
}
```



DaoAuthenticationProvider

Recibe el `UserDetails` y usa `PasswordEncoder.matches()` para comparar hashes.

Configuración de SecurityConfig: Filter Chain y Form Login

Configuración Declarativa

Reemplazamos el login generado por defecto configurando nuestra propia página de inicio de sesión:



loginPage("/login")

Ruta del controlador Spring MVC que renderiza la vista custom.



defaultSuccessUrl("/usuarios", true)

Página a la que se redirige de forma obligatoria tras autenticación exitosa.



permitAll() en Login & Estáticos

Garantiza que el formulario de login y los assets estáticos (/css, /js) sean accesibles sin sesión previa.

```
@Configuration
@EnableWebSecurity
public class SecurityConfig {

    @Bean
    public SecurityFilterChain filterChain(HttpSecurity http)
        throws Exception {
        return http
            .authorizeHttpRequests(auth -> auth
                .requestMatchers("/login", "/css/**", "/js/**").permitAll()
                .anyRequest().authenticated()
            )
            .formLogin(form -> form
                .loginPage("/login")
                .defaultSuccessUrl("/usuarios", true)
                .permitAll()
            )
            .logout(logout -> logout
                .logoutSuccessUrl("/login?logout").permitAll()
            )
            .build();
    }
}
```

Controlador y Vista de Login Personalizado

LoginController.java (Spring MVC)

Controlador simple para despachar la plantilla Thymeleaf `login.html` :

```
@Controller
public class LoginController {

    @GetMapping("/login")
    public String login() {
        return "login"; //Retorna login.html
    }
}
```

Nota: No se requiere un `@PostMapping("/login")` porque Spring Security intercepta el POST automáticamente con `UsernamePasswordAuthenticationFilter` .

login.html (Thymeleaf con CSRF)

```
<!-- src/main/resources/templates/login.html -->
<form th:action="@{/login}" method="post" class="login-box">
    <!-- Alerta de error si las credenciales fallan -->
    <div th:if="${param.error}" class="alert-danger">
        Correo o contraseña incorrectos.
    </div>

    <label>Correo Institucional</label>
    <input type="text" name="username" required />

    <label>Contraseña</label>
    <input type="password" name="password" required />

    <!-- th:action inyecta automáticamente el token_csrf -->
    <button type="submit">Iniciar Sesión</button>
</form>
```

Live Demo 2: SecurityConfig, Login Propio y Depuración

Actividad en Vivo: SecurityConfig y Filtros



1. Crear SecurityConfig & In-Memory User

Declarar `SecurityFilterChain` con `.loginPage("/login")` y usuario de prueba en memoria.



2. Crear LoginController & login.html

Añadir `@GetMapping("/login")` y diseñar la vista Thymeleaf con soporte de errores y CSRF.



3. Activar Logs de Seguridad

En `application.properties`: `logging.level.org.springframework.security=DEBUG`



4. Probar Flujo Completo & Consola

Navegar a `/usuarios` → Redirección a `/login` → Autenticar → Analizar filtros en consola.

// Salida real de Spring Boot Console

DEBUG o.s.s.web.DefaultSecurityFilterChain:

Will secure any request with [

```
DisableEncodeUrlFilter,  
SecurityContextHolderFilter,  
HeaderWriterFilter,  
CsrfFilter,  
LogoutFilter,  
UsernamePasswordAuthenticationFilter,  
DefaultLoginPageGeneratingFilter,  
ExceptionTranslationFilter,  
AuthorizationFilter
```

]

Fin de la Sesión 1

Próxima Sesión: Autorización Granular RBAC,
Persistencia JPA en BD y Vistas Thymeleaf